

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA Nº 09-1

NOMBRE COMPLETO: Antonio Manuel Zamora Ayala

Nº de Cuenta: 320152896

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 24 de octubre de 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1. **Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.**

1.1 Una nave espacial o nave de carreras sigue el circuito de la carrera con rectas, curvas, rampas de subida y bajada.

Ya que mi equipo hizo la pista de carreras por código, me apoyé de la herramienta *Microsoft Copilot* para implementar la animación de la navecita recorriendo la pista de carreras.

P09_320152896.cpp modifiqué los bloques de código siguientes:

- Carga del modelo:

```
76    // Nave espacial
77    Model naveEspacial;
```

```
493       //Modelo de la nave espacial
494       naveEspacial = Model();
495       naveEspacial.LoadModel("Models/naveEspacial.obj");
```

- Dibujo de la nave:

```
92    // Variables para la nave
93
94    float movNave = 0.0f;
95    float movNaveOffset = 0.8f;
96
97    int estadoNave = 0;
98
99    float rotacionNaveY = 0.0f; // dirección
100   float rotacionNaveZ = 0.0f; // inclinación lateral
101   float rotacionNaveX = 0.0f; // inclinación al subir/bajar
102
103   float avanceNave = 0.0f;
104   float velocidadNave = 0.4f;
```

```
159   // Nave espacial
160   Model naveEspacial;
```

```
516     //Modelo de la nave espacial
517     naveEspacial = Model();
518     naveEspacial.LoadModel("Models/naveEspacial.obj");
```

```
554     // Luz morada para la nave
555     pointLights[2] = PointLight(1.0f, 0.0f, 1.0f,
556         0.8f, 1.0f,
557         0.0f, 0.0f, 0.0f,
558         0.3f, 0.2f, 0.1f);
559     pointLightCount++;
560
561     // Luz azul para el motor 1 de la nave
562     pointLights[3] = PointLight(0.0f, 0.0f, 1.0f,
563         1.0f, 1.0f,             // radiacion y color (0.3f y 0.6f)
564         0.0f, 0.0f, 0.0f,
565         0.1f, 0.1f, 0.03f);
566     pointLightCount++;
567
568     // Luz azul para el motor 2 de la nave
569     pointLights[4] = PointLight(0.0f, 0.0f, 1.0f,
570         1.0f, 1.0f,             // radiacion y color (0.3f y 0.6f)
571         0.0f, 0.0f, 0.0f,
572         0.1f, 0.1f, 0.03f);
573     pointLightCount++;
```

```
612     // 4. Faro frontal de la lave (índice 4)
613     spotLights[4] = SpotLight(1.0f, 1.0f, 1.0f,
614         1.0f, 1.0f,
615         0.0f, 2.8f, 0.0f,
616         1.0f, 0.0f, 0.0f, // Dirección hacia adelante (+X)
617         1.0f, 0.0005f, 0.00019f,
618         25.0f); // Apertura del faro
619     spotLightCount++;
```

```
889         // Nave espacial
890
891         movNave += movNaveOffset * deltaTime;
892         avanceNave += velocidadNave * deltaTime;
893
894         if (avanceNave >= raceTrack.pathData.size())
895         {
896             avanceNave = 0.0f;
897         }
```

```
959         // Luz morada de la nave (indice 2 original)
960         if (mainWindow.getTeclaNavePresionada()) {
961             lucesActivas[lucesActivasCount++] = pointLights[2];
962         }
963
964         if (mainWindow.getTeclaMotor1())
965         {
966             lucesActivas[lucesActivasCount++] = pointLights[3];
967         }
968
969         if (mainWindow.getTeclaMotor2())
970         {
971             lucesActivas[lucesActivasCount++] = pointLights[4];
972         }
```

```
1369        // Nave espacial
1370
1371        model = glm::mat4(1.0f);
1372
1373        int idx = (int)avanceNave;
1374
1375        if (idx >= raceTrack.pathData.size())
1376        {
1377            idx = 0;
1378        }
1379
1380        PlanetaryRaceTrack::PathSample sample = raceTrack.pathData[idx];
1381
1382        // La pista está desplazada -250 en Z
1383        glm::vec3 posicionNave = sample.pos +glm::vec3(0.0f, 3.0f, -250.0f);
1384
1385        glm::vec3 forward = glm::normalize(sample.fwd);
1386        glm::vec3 up = glm::normalize(sample.up);
1387        glm::vec3 right = glm::normalize(sample.right);
1388
1389        // Construcción de matriz de orientación usando los ejes de la pista
1390        glm::mat4 orientacion(1.0f);
1391
1392        orientacion[0] = glm::vec4(right, 0.0f);
1393        orientacion[1] = glm::vec4(up, 0.0f);
1394        orientacion[2] = glm::vec4(-forward, 0.0f);
1395
1396        model = glm::translate(glm::mat4(1.0f),posicionNave);
1397
1398        model *= orientacion;
1399
1400        // Ajustes propios del modelo
1401        model = glm::rotate(model,glm::radians(180.0f),glm::vec3(0.0f, 1.0f, 0.0f));
1402
1403        // MATRIZ PARA LUCES
1404        glm::mat4 naveMatrixLuces = model;
1405
1406        model = glm::scale(model,glm::vec3(7.0f, 7.0f, 7.0f));
1407
1408        // ================= LUCES DE LA NAVE =================
1409
1410        glm::vec3 navePuntualPos =glm::vec3(naveMatrixLuces * glm::vec4(0.0f, 2.25f, -3.0f, 1.0f));
1411
1412        pointLights[2].SetPosition(navePuntualPos);
1413
1414        glm::vec3 naveSpotPos = glm::vec3( naveMatrixLuces * glm::vec4(0.0f, 0.0f, 5.0f, 1.0f));
1415
1416        glm::vec3 naveSpotDir = glm::normalize( glm::vec3( naveMatrixLuces * glm::vec4(0.0f, -0.4f, 1.0f, 0.0f)));
1417
1418        spotLights[4].SetFlash(naveSpotPos,naveSpotDir);
```

```
1420        // Motor izquierdo
1421        glm::vec3 motor1Pos =glm::vec3(naveMatrixLuces *glm::vec4(-7.35f, 3.0f, -14.5f, 1.0f));
1422
1423        pointLights[3].SetPosition(motor1Pos);
1424
1425        // Motor derecho
1426        glm::vec3 motor2Pos = glm::vec3( naveMatrixLuces * glm::vec4(7.5f, 3.0f, -14.5f, 1.0f));
1427
1428        pointLights[4].SetPosition(motor2Pos);
1429
1430        // ================== RENDER ==================
1431
1432        glUniformMatrix4fv(uniformModel,1,GL_FALSE,glm::value_ptr(model));
1433
1434        color = glm::vec3(1.0f, 1.0f, 1.0f);
1435
1436        glUniform3fv(uniformColor,1,glm::value_ptr(color));
1437
1438        naveEspacial.RenderModel();
```

- Dibujo de la pista:

```
34  #include "PlanetaryRaceTrack.h"
```

```
186 // ---- SHADERS PARA LA PISTA ----
187 const char* vCodeTrack = R"(
188 #version 330 core
189 layout (location = 0) in vec3 inPos;
190 layout (location = 1) in vec2 inUV;
191 layout (location = 2) in vec3 inNorm;
192
193 out vec2 uvData;
194 out vec3 normData;
195 out vec3 fragWorldPos;
196
197 uniform mat4 mMat;
198 uniform mat4 vMat;
199 uniform mat4 pMat;
200
201 void main() {
202     fragWorldPos = vec3(mMat * vec4(inPos, 1.0));
203     normData = mat3(transpose(inverse(mMat))) * inNorm;
204     uvData = inUV;
205     gl_Position = pMat * vMat * vec4(fragWorldPos, 1.0);
206 }
207 )";
208
209 const char* fCodeTrack = R"(
210 #version 330 core
211 out vec4 finalColor;
212
213 in vec2 uvData;
214 in vec3 normData;
215 in vec3 fragWorldPos;
216
217 void main() {
218     vec3 lDir = normalize(vec3(0.5, 1.0, 0.4));
219     float nDotL = max(dot(normalize(normData), lDir), 0.25);
220
221     vec3 albedo;
222     if (uvData.x >= 0.19 && uvData.x <= 0.81) {
223         albedo = vec3(0.20, 0.22, 0.25);
224         if (abs(uvData.x - 0.5) < 0.015 && fract(uvData.y * 0.5) < 0.5) {
225             albedo = vec3(0.95, 0.80, 0.15);
226         }
227     } else {
228         float st = step(0.5, fract(uvData.y * 0.5));
229         albedo = mix(vec3(0.85, 0.15, 0.15), vec3(0.92, 0.92, 0.92), st);
230     }
231     finalColor = vec4(albedo * nDotL, 1.0);
232 }
233 )";
234 // ------------------------------
```

```
403     // ---- INICIALIZAR PISTA DE CARRERAS ----
404     GLuint vsTrack = glCreateShader(GL_VERTEX_SHADER);
405     glShaderSource(vsTrack, 1, &vCodeTrack, NULL);
406     glCompileShader(vsTrack);
407     GLuint fsTrack = glCreateShader(GL_FRAGMENT_SHADER);
408     glShaderSource(fsTrack, 1, &fCodeTrack, NULL);
409     glCompileShader(fsTrack);
410     GLuint trackProg = glCreateProgram();
411     glAttachShader(trackProg, vsTrack);
412     glAttachShader(trackProg, fsTrack);
413     glLinkProgram(trackProg);
414
415     PlanetaryRaceTrack raceTrack;
416     raceTrack.buildMesh();
417     // ---------------------------------------
```

```
911         // ---- RENDER PISTA DE CARRERAS ----
912         glUseProgram(trackProg);
913         glm::mat4 trackModel = glm::mat4(1.0f);
914
915         // 1. PRIMERO mueve la pista a la coordenada deseada
916         trackModel = glm::translate(trackModel, glm::vec3(0.0f, 0.0f, -250.0f));
917
918
919         glUniformMatrix4fv(glGetUniformLocation(trackProg, "pMat"), 1, GL_FALSE, glm::value_ptr(projection));
920         glUniformMatrix4fv(glGetUniformLocation(trackProg, "vMat"), 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
921         glUniformMatrix4fv(glGetUniformLocation(trackProg, "mMat"), 1, GL_FALSE, glm::value_ptr(trackModel));
922         raceTrack.draw();
923         // ---------------------------------
```

También modifiqué líneas de código en *PlanetaryRaceTrack.h*:

- *PlanetaryRaceTrack.h*:

```cpp
#pragma once

#include <vector>
#include <cmath>
#include <glew.h>
#include <glm.hpp>
#include <gtc\matrix_transform.hpp>

struct TrackPoint {
    glm::vec3 position;
    float bankingAngle;
    bool isJump;
};

class PlanetaryRaceTrack {
public:
    GLuint vaoID = 0, vboID = 0, eboID = 0;
    GLsizei totalIndices = 0;

    std::vector<TrackPoint> controlPoints;
    int smoothSteps = 25;
    float width = 10.0f;
    float wallH = 0.8f;
    float wallW = 0.6f;

    struct PathSample {
        glm::vec3 pos;
        glm::vec3 fwd;
        glm::vec3 up;
        glm::vec3 right;
    };
    std::vector<PathSample> pathData;

    PlanetaryRaceTrack() {
        setupDefaultTrack();
    }

    ~PlanetaryRaceTrack() {
        if (eboID) glDeleteBuffers(1, &eboID);
        if (vboID) glDeleteBuffers(1, &vboID);
        if (vaoID) glDeleteVertexArrays(1, &vaoID);
    }

    void setupDefaultTrack() {
        controlPoints = {
            { glm::vec3(0.0f,  0.3f,   0.0f),   0.0f, false },
            { glm::vec3(55.0f,  0.3f,   0.0f),   0.0f, false },
            { glm::vec3(95.0f,  4.5f,  30.0f),  12.0f, false },
            { glm::vec3(105.0f, 14.0f,  85.0f),   8.0f, false },
            { glm::vec3(65.0f, 18.0f, 130.0f),  15.0f, false },
            { glm::vec3(15.0f, 23.0f, 135.0f),   0.0f, true },
            { glm::vec3(-35.0f,  8.0f, 125.0f),   0.0f, false },
            { glm::vec3(-80.0f,  2.5f,  95.0f), -12.0f, false },
            { glm::vec3(-55.0f,  1.5f,  55.0f),  12.0f, false },
            { glm::vec3(-85.0f,  0.5f,  15.0f), -15.0f, false },
            { glm::vec3(-45.0f,  0.3f,  -5.0f),   0.0f, false }
        };
    }
```

```
    static glm::vec3 interpolatePos(const glm::vec3& p0, const glm::vec3& p1, const glm::vec3& p2, const glm::vec3& p3, float t) {
        float tSq = t * t;
        float tCub = tSq * t;
        return 0.5f * ((2.0f * p1) + (-p0 + p2) * t + (2.0f * p0 - 5.0f * p1 + 4.0f * p2 - p3) * tSq + (-p0 + 3.0f * p1 - 3.0f * p2 + p3) * tCub);
    }

    static glm::vec3 interpolateDir(const glm::vec3& p0, const glm::vec3& p1, const glm::vec3& p2, const glm::vec3& p3, float t) {
        float tSq = t * t;
        return glm::normalize(0.5f * ((-p0 + p2) + 2.0f * (2.0f * p0 - 5.0f * p1 + 4.0f * p2 - p3) * t + 3.0f * (-p0 + 3.0f * p1 - 3.0f * p2 + p3) * tSq));
    }

    void buildMesh() {
        std::vector<float> vData;
        std::vector<unsigned int> iData;
        pathData.clear();

        int nPts = static_cast<int>(controlPoints.size());
        int totalFrames = nPts * smoothSteps;
        float uvMult = 0.20f;
        const int vertsPerSlice = 6;

        for (int i = 0; i <= totalFrames; ++i) {
            int segment = (i / smoothSteps) % nPts;
            float t = static_cast<float>(i % smoothSteps) / smoothSteps;
            if (i == totalFrames) { segment = 0; t = 0.0f; }

            const TrackPoint& n0 = controlPoints[(segment - 1 + nPts) % nPts];
            const TrackPoint& n1 = controlPoints[segment];
            const TrackPoint& n2 = controlPoints[(segment + 1) % nPts];
            const TrackPoint& n3 = controlPoints[(segment + 2) % nPts];

            glm::vec3 cPos = interpolatePos(n0.position, n1.position, n2.position, n3.position, t);
            glm::vec3 cDir = interpolateDir(n0.position, n1.position, n2.position, n3.position, t);

            glm::vec3 globalUp(0.0f, 1.0f, 0.0f);
            glm::vec3 cRight = glm::normalize(glm::cross(cDir, globalUp));
            glm::vec3 cNorm = glm::normalize(glm::cross(cRight, cDir));

            float bAngle = glm::radians(glm::mix(n1.bankingAngle, n2.bankingAngle, t));
            glm::mat4 bRot = glm::rotate(glm::mat4(1.0f), bAngle, cDir);
            cRight = glm::normalize(glm::vec3(bRot * glm::vec4(cRight, 0.0f)));
            cNorm = glm::normalize(glm::vec3(bRot * glm::vec4(cNorm, 0.0f)));

            if (i < totalFrames) pathData.push_back({ cPos, cDir, cNorm, cRight });
```

```
            float hw = width * 0.5f;
            float vc = static_cast<float>(i) * uvMult;

            glm::vec3 pRL = cPos - cRight * hw;
            glm::vec3 pRR = cPos + cRight * hw;
            glm::vec3 pWL = pRL - cRight * wallW + cNorm * wallH;
            glm::vec3 pWR = pRR + cRight * wallW + cNorm * wallH;
            glm::vec3 pBL = pRL - cRight * wallW - cNorm * 0.8f;
            glm::vec3 pBR = pRR + cRight * wallW - cNorm * 0.8f;

            glm::vec3 pts[vertsPerSlice] = { pBL, pWL, pRL, pRR, pWR, pBR };
            float uvs[vertsPerSlice] = { 0.0f, 0.1f, 0.2f, 0.8f, 0.9f, 1.0f };
            glm::vec3 nms[vertsPerSlice] = { -cRight, cNorm, cNorm, cNorm, cNorm, cRight };

            for (int k = 0; k < vertsPerSlice; ++k) {
                vData.push_back(pts[k].x); vData.push_back(pts[k].y); vData.push_back(pts[k].z);
                vData.push_back(uvs[k]);   vData.push_back(vc);
                vData.push_back(nms[k].x); vData.push_back(nms[k].y); vData.push_back(nms[k].z);
            }
        }

        for (int i = 0; i < totalFrames; ++i) {
            int segment = (i / smoothSteps) % nPts;
            int localI = i % smoothSteps;

            if (controlPoints[segment].isJump && localI > (smoothSteps * 0.65f)) continue;

            unsigned int bCurr = i * vertsPerSlice;
            unsigned int bNext = (i + 1) * vertsPerSlice;

            for (int q = 0; q < vertsPerSlice - 1; ++q) {
                iData.push_back(bCurr + q); iData.push_back(bCurr + q + 1); iData.push_back(bNext + q);
                iData.push_back(bCurr + q + 1); iData.push_back(bNext + q + 1); iData.push_back(bNext + q);
            }
        }

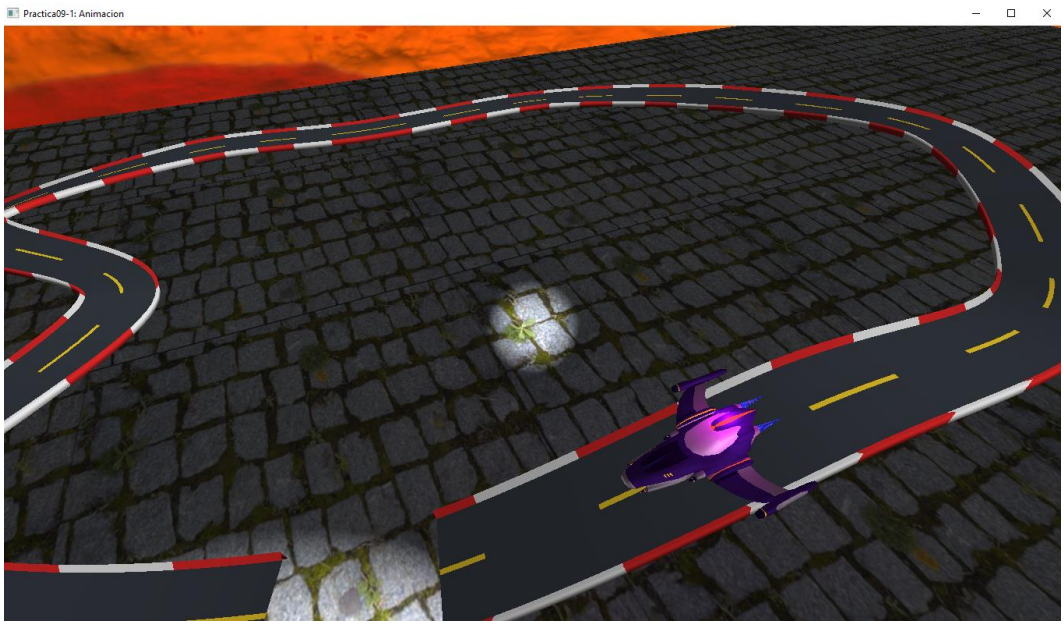
        totalIndices = static_cast<GLsizei>(iData.size());

        glGenVertexArrays(1, &vaoID);
        glGenBuffers(1, &vboID);
        glGenBuffers(1, &eboID);

        glBindVertexArray(vaoID);
        glBindBuffer(GL_ARRAY_BUFFER, vboID);
        glBufferData(GL_ARRAY_BUFFER, vData.size() * sizeof(float), vData.data(), GL_STATIC_DRAW);
        glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, eboID);
        glBufferData(GL_ELEMENT_ARRAY_BUFFER, iData.size() * sizeof(unsigned int), iData.data(), GL_STATIC_DRAW);
```

```
152
153         GLsizei st = 8 * sizeof(float);
154         glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, st, (void*)0);
155         glEnableVertexAttribArray(0);
156         glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, st, (void*)(3 * sizeof(float)));
157         glEnableVertexAttribArray(1);
158         glVertexAttribPointer(2, 3, GL_FLOAT, GL_FALSE, st, (void*)(5 * sizeof(float)));
159         glEnableVertexAttribArray(2);
160         glBindVertexArray(0);
161     }
162
163     void draw() const {
164         glBindVertexArray(vaoID);
165         glDrawElements(GL_TRIANGLES, totalIndices, GL_UNSIGNED_INT, 0);
166         glBindVertexArray(0);
167     }
168 };
```

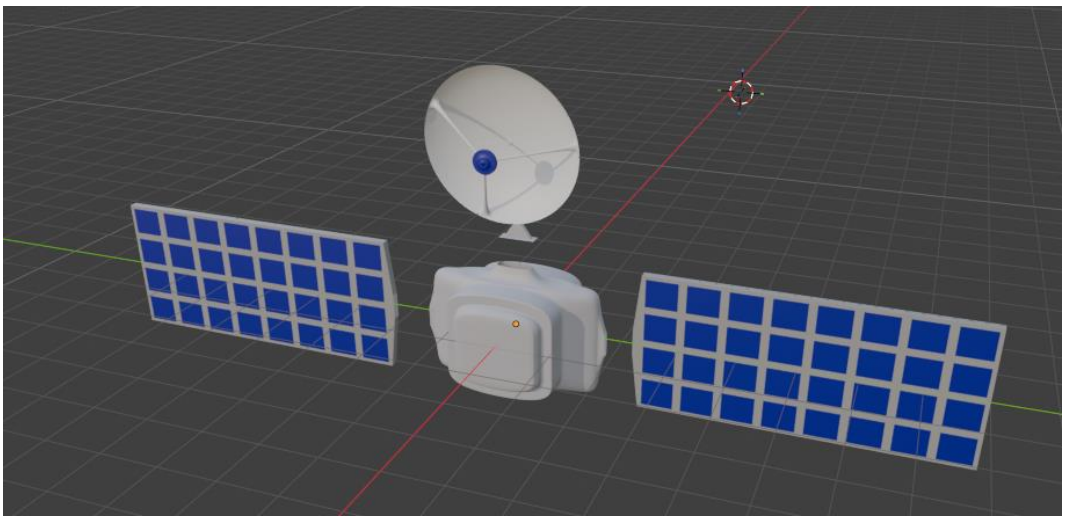
Agrego captura de pantalla de la nave:



NOTA1 Sólo se agrega una captura de pantalla muy general, ya que todo se muestra en el video.

1.2 Satélite vuela siguiendo alguna trayectoria y la animación de sus componentes que previamente era con teclado.

Primero texturicé en Blender mi modelo del satélite que había ocupado para la práctica de jerarquía y después exporté cada elemento por separado:



En *P09_320152896.cpp* modifiqué los bloques de código siguientes:

- Carga de modelo:

```
// Variables para el satelite

float movSatelite = 0.0f;
float movSateliteOffset = 0.4f;

int estadoSatelite = 0;
float rotacionSatelite = 0.0f;

float rotaAntena = 0.0f;
float rotaAntenaOffset = 0.025f;

float rotaPanelIzq = 0.0f;
float rotaPanelDer = 0.0f;

float rotaPanelIzqOffset = 0.01f;
float rotaPanelDerOffset = 0.01f;
```

```
133    // Modelo del satélite
134    Model cuerpoSat;
135    Model antenaSat;
136    Model panelDerSat;
137    Model panelIzqSat;
```

```
416    // Modelo del satélite
417    cuerpoSat = Model();
418    cuerpoSat.LoadModel("Models/cuerpoSat.obj");
419
420    antenaSat = Model();
421    antenaSat.LoadModel("Models/antena.obj");
422
423    panelIzqSat = Model();
424    panelIzqSat.LoadModel("Models/panelIzq.obj");
425
426    panelDerSat = Model();
427    panelDerSat.LoadModel("Models/panelDer.obj");
```

- Animación básica: Ya que aún no podemos implementar tal cual movimientos circulares (como la rotación de un satélite en un planeta), opté por hacer un recorrido en forma de decágono (simulando un círculo). Me base en un switch como lo que hice para el avioncito, pero agregué más casos.

```
543        glm::mat4 modelaux4(1.0); // Para el satélite
544        glm::mat4 modelaux5(1.0); // Para la jerarquia del satélite
```

```
670         // Para el satélite
671
672         rotaAntena += rotaAntenaOffset * deltaTime;
673         rotaPanelIzq += rotaPanelIzqOffset * deltaTime;
674         rotaPanelDer += rotaPanelDerOffset * deltaTime;
675
676         movSatelite += movSateliteOffset * deltaTime;
677
678         switch (estadoSatelite)
679         {
680         case 0:
681             rotacionSatelite = 0.0f;
682             if (movSatelite >= 60.0f)
683             {
684                 movSatelite = 0.0f;
685                 estadoSatelite = 1;
686             }
687             break;
688
689         case 1:
690             rotacionSatelite = -36.0f;
691             if (movSatelite >= 60.0f)
692             {
693                 movSatelite = 0.0f;
694                 estadoSatelite = 2;
695             }
696             break;
697
698         case 2:
699             rotacionSatelite = -72.0f;
700             if (movSatelite >= 60.0f)
701             {
702                 movSatelite = 0.0f;
703                 estadoSatelite = 3;
704             }
705             break;
706
707         case 3:
708             rotacionSatelite = -108.0f;
709             if (movSatelite >= 60.0f)
710             {
711                 movSatelite = 0.0f;
712                 estadoSatelite = 4;
713             }
714             break;
```

```
715
716             case 4:
717                 rotacionSatelite = -144.0f;
718                 if (movSatelite >= 60.0f)
719                 {
720                     movSatelite = 0.0f;
721                     estadoSatelite = 5;
722                 }
723                 break;
724
725             case 5:
726                 rotacionSatelite = -180.0f;
727                 if (movSatelite >= 60.0f)
728                 {
729                     movSatelite = 0.0f;
730                     estadoSatelite = 6;
731                 }
732                 break;
733
734             case 6:
735                 rotacionSatelite = -216.0f;
736                 if (movSatelite >= 60.0f)
737                 {
738                     movSatelite = 0.0f;
739                     estadoSatelite = 7;
740                 }
741                 break;
742
743             case 7:
744                 rotacionSatelite = -252.0f;
745                 if (movSatelite >= 60.0f)
746                 {
747                     movSatelite = 0.0f;
748                     estadoSatelite = 8;
749                 }
750                 break;
751
752             case 8:
753                 rotacionSatelite = -288.0f;
754                 if (movSatelite >= 60.0f)
755                 {
756                     movSatelite = 0.0f;
757                     estadoSatelite = 9;
758                 }
759                 break;
760
761             case 9:
762                 rotacionSatelite = -324.0f;
763                 if (movSatelite >= 60.0f)
764                 {
765                     movSatelite = 0.0f;
766                     estadoSatelite = 0;
767                 }
768                 break;
769             }
```

```
        // Satelite
        // Cuerpo del satelite
        model = modelaux4;
        color = glm::vec3(1.0f, 1.0f, 1.0f);
        model = glm::mat4(1.0);

        switch (estadoSatelite)
        {
        case 0:
            model = glm::translate(model, glm::vec3(-30.0f + movSatelite, 45.0f, 0.0f));
            break;

        case 1:
            model = glm::translate(model, glm::vec3(30.0f + movSatelite * 0.809f, 45.0f, movSatelite * 0.588f));
            break;

        case 2:
            model = glm::translate(model, glm::vec3(78.5f + movSatelite * 0.309f, 45.0f, 35.3f + movSatelite * 0.951f));
            break;

        case 3:
            model = glm::translate(model, glm::vec3(97.0f - movSatelite * 0.309f, 45.0f, 92.4f + movSatelite * 0.951f));
            break;

        case 4:
            model = glm::translate(model, glm::vec3(78.5f - movSatelite * 0.809f, 45.0f, 149.5f + movSatelite * 0.588f));
            break;

        case 5:
            model = glm::translate(model, glm::vec3(30.0f - movSatelite, 45.0f, 184.8f));
            break;

        case 6:
            model = glm::translate(model, glm::vec3(-30.0f - movSatelite * 0.809f, 45.0f, 184.8f - movSatelite * 0.588f));
            break;

        case 7:
            model = glm::translate(model, glm::vec3(-78.5f - movSatelite * 0.309f, 45.0f, 149.5f - movSatelite * 0.951f));
            break;

        case 8:
            model = glm::translate(model, glm::vec3(-97.0f + movSatelite * 0.309f, 45.0f, 92.4f - movSatelite * 0.951f));
            break;

        case 9:
            model = glm::translate(model, glm::vec3(-78.5f + movSatelite * 0.809f, 45.0f, 35.3f - movSatelite * 0.588f));
            break;
        }

        model = glm::rotate(model, glm::radians(rotacionSatelite), glm::vec3(0.0f, 1.0f, 0.0f));

        model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 1.0f, 0.0f));

        model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
        modelaux = model;
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        cuerpoSat.RenderModel();
        // Antena
        model = modelaux;                       //  z , x, y
        model = glm::translate(model, glm::vec3(0.0f, 0.0f, -0.75f));
        model = glm::rotate(model, rotaAntena, glm::vec3(0.0f, 0.0f, 1.0f));
        modelaux5 = model;
        model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        antenaSat.RenderModel();

        // Panel solar 1
        model = modelaux;
        model = glm::translate(model, glm::vec3(0.0f, 1.7f, 0.0f));
        model = glm::rotate(model, rotaPanelIzq, glm::vec3(0.0f, 1.0f, 0.0f));
        modelaux5 = model;
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        panelIzqSat.RenderModel();

        // Panel solar 2
        model = modelaux;
        model = glm::translate(model, glm::vec3(0.0f, -1.65f, 0.0f));
        model = glm::rotate(model, rotaPanelDer, glm::vec3(0.0f, 1.0f, 0.0f));
        modelaux5 = model;
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        panelDerSat.RenderModel();
```

NOTA2 No se agregan capturas de pantalla, todo se muestra en el video.

2. **Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla.**

   Para esta práctica no se presentaron problemas.

3. **Conclusión:**
   a. **Los ejercicios del reporte:** Me gustó poder animar ya algunos de los modelos con los que hemos trabajado a lo largo del semestre, ya que esta es la última etapa para antes del proyecto.

   b. **Comentarios generales:** Considero que fueron ejercicios al nivel de lo que se enseñó en el laboratorio y sirvió para reforzar lo relacionado a la animación básica

   c. **Conclusión.** Esta práctica me sirvió mucho para saber cómo funciona de una mejor manera la animación en OpenGL. Noté que para la pista de carreras usé algo de animación compleja, pero la verdad por el tema del tiempo y demás cosas, no pude dedicarme al 100% para lograr la implementación con banderas. Finalmente esto me ayuda mucho para saber como satisfacer los requerimientos del proyecto final.

Referencias bibliográficas.

https://sketchfab.com/3d-models/multi-universe-space-ship-3d-model-42a2cbc97d9c489b80c6533c44294d55